

Программирование на языке Python

Программирование на языке Python

**Простейшие программы.
Ввод и вывод данных.**

Простейшая программа

```
# Это пустая программа
```

комментарии после #
не обрабатываются

Вывод на экран

```
print ( "2+2=?" )  
print ( "Ответ: 4" )
```

автоматический
переход на новую
строку

Протокол:

2+2=?

Ответ: 4

```
print ( ' 2+2=? ' )  
print ( ' Ответ: 4 ' )
```



Кавычки могут быть и одинарными, и двойными.

Вывод на экран

Чтобы Python мог правильно распознать текст, пользуемся разными кавычками:

- ▶ если в тексте нужны одинарные кавычки, то для обрамления такого текста используем двойные кавычки;
- ▶ если в тексте нужны двойные кавычки, то обрамляем его одинарными.

```
print('В тексте есть "двойные" кавычки')  
print("В тексте есть 'одинарные' кавычки")
```

Протокол вывода:

В тексте есть "двойные" кавычки

В тексте есть 'одинарные' кавычки

Вывод на экран



А что, если мы хотим вывести на экран такую строку: Dragon's mother said "No"

Чтобы указать интерпретатору какую кавычку считать частью строки, а не началом или концом строки, используют символ экранирования: \ (обратный слэш).

```
print("Dragon's mother said \"No\"")
```

Протокол вывода:

Dragon's mother said "No"

Необязательные параметры команды print

Параметр sep

(separator, разделитель)

```
print('a', 'b', 'c', sep='*')  
print('d', 'e', 'f', sep='**')
```

Протокол вывода:

a*b*c

d**e**f

Необязательные параметры команды print

Параметр end

Если перевод строки делать не нужно или требуется указать специальное окончание, то следует явно указать значение для параметра end.

```
print('a', 'b', 'c', end='@')  
print('d', 'e', 'f', end='@@')
```

Протокол вывода:

a b c@d e f@@

Примечания

Примечание 1. Вызов команды `print()` с пустыми скобками ставит перевод строки.

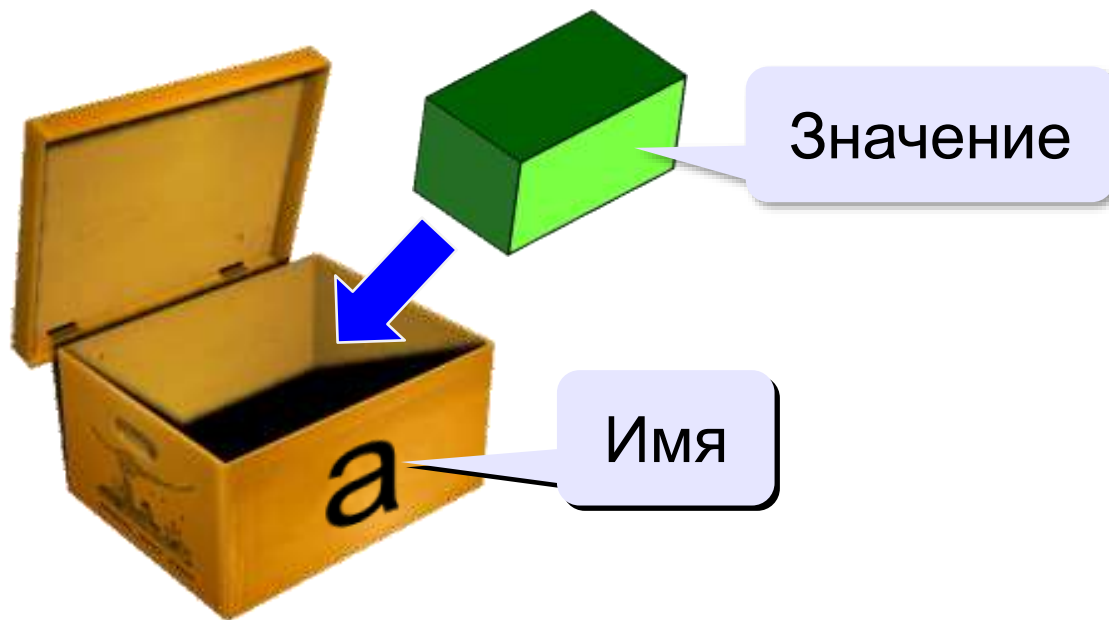
Примечание 2. Последовательность символов `\n` называется управляющей последовательностью и задает перевод строки.

Примечание 3. Если после вывода данных нужно более одного перевода строки, то необходимо использовать следующий код:

```
print('Python', end='\n\n\n')
```

Переменные

Переменная – это величина, имеющая имя, тип и значение. Значение переменной можно изменять во время работы программы.



Имена переменных

МОЖНО использовать

- латинские буквы (A-Z, a-z)

заглавные и строчные буквы **различаются**

- русские буквы (**не рекомендуется!**)
- цифры

имя не может начинаться с цифры

- знак подчеркивания _

НЕЛЬЗЯ использовать

- ~~скобки~~
- ~~знаки +, =, !, ? и др.~~

Какие имена правильные?

AXby R&B 4Wheel Вася “PesBarbos”
TU154 [QuQu] _ABBA A+B

Как записать значение в переменную?

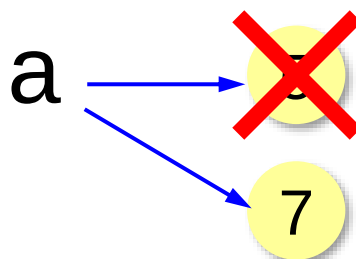
оператор
присваивания



При записи нового значения
старое удаляется из памяти!

a = 5

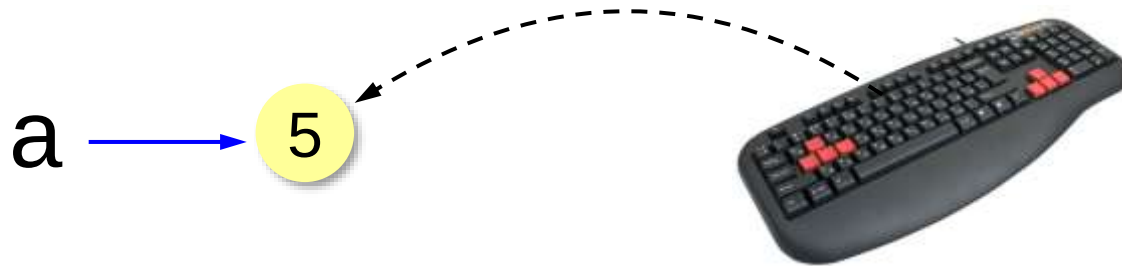
a = 7



Оператор – это команда языка программирования (инструкция).

Оператор присваивания – это команда для записи нового значения переменной.

Ввод значения с клавиатуры



1. Программа ждет, пока пользователь введет значение и нажмет *Enter*.
2. Введенное значение записывается в переменную **a** (связывается с именем **a**)

Ввод значения с клавиатуры

```
a = input ()
```

ввести строку с клавиатуры
и связать с переменной `a`

```
b = input ()
```

```
c = a + b
```

```
print ( c )
```

Протокол:

21

33

2133



Почему?



Результат функции `input` – строка символов!

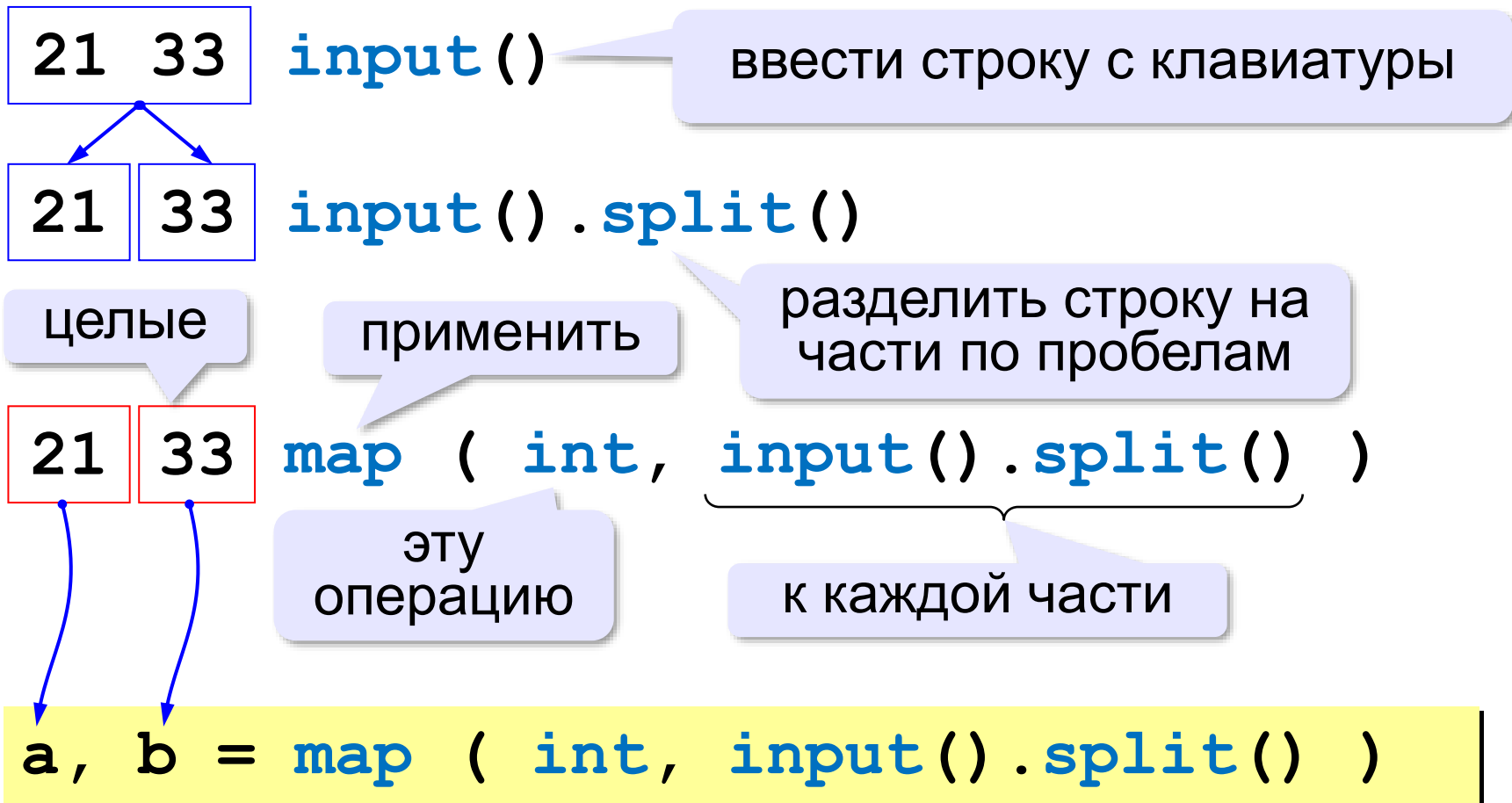
преобразовать в
целое число

```
a = int ( input () )
```

```
b = int ( input () )
```

Ввод двух значений в одной строке

```
a, b = map ( int, input().split() )
```



Ввод с подсказкой

```
a = input ( "Введите число: " )
```

Введите число: 26

подсказка



Что не так?

```
a = int( input("Введите число: ") )
```

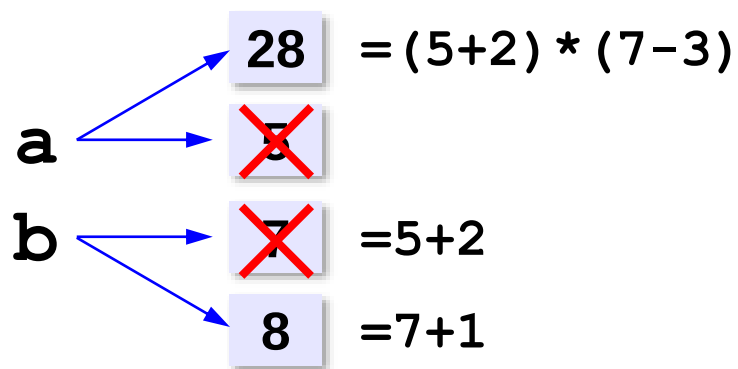

Изменение значений переменной

$a = 5$

$b = a + 2$

$a = (a + 2) * (b - 3)$

$b = b + 1$



Вывод данных

```
print ( a )
```

значение
переменной

```
print ( "Ответ: ", a )
```

значение и
текст

перечисление через запятую

```
print ( "Ответ: ", a+b )
```

вычисление
выражения

```
print ( a, "+", b, "=", c )
```

2 + 3 = 5

через пробелы

```
print ( a, "+", b, "=", c, sep = " " )
```

2+3=5

убрать разделители

Сложение чисел: простое решение

```
a = int ( input () )  
b = int ( input () )  
c = a + b  
print ( c )
```



Что плохо?

Сложение чисел: полное решение

```
print ( "Введите два числа: " )  
a = int ( input() )  
b = int ( input() )  
c = a + b  
print ( a, "+", b, "=", c )
```

подсказка

Протокол:

компьютер

Введите два числа

25 30

пользователь

25 + 30 = 55

Программирование на языке Python

Вычисления

Типы данных

- `int` # целое
- `float` # вещественное
- `bool` # логические значения
- `str` # символьная строка

```
a = 5
```

```
print ( type(a) )
```

```
a = 4.5
```

```
print ( type(a) )
```

```
a = True
```

```
print ( type(a) )
```

```
a = "Петя"
```

```
print ( type(a) )
```

```
<class 'int'>
```

```
<class 'float'>
```

```
<class 'bool'>
```

```
<class 'str'>
```

Зачем нужен тип переменной?

Тип определяет:

- область допустимых значений
- допустимые операции
- объём памяти
- формат хранения данных

Арифметическое выражения

3 1 2 4 5 6

```
a = (c + b**5*3 - 1) / 2 * d
```

Приоритет (*старшинство*):

- 1) скобки
- 2) возведение в степень **
- 3) умножение и деление
- 4) сложение и вычитание

$$a = \frac{c + b^5 \cdot 3 - 1}{2} \cdot d$$

```
a = (c + b*5*3 - 1) \
      / 2 * d
```

перенос на
следующую строку

```
a = (c + b*5*3
      - 1) / 2 * d
```

перенос внутри
скобок разрешён

Деление

Классическое деление:

```
a = 9; b = 6
x = 3 / 4    # = 0.75
x = a / b    # = 1.5
x = -3 / 4   # = -0.75
x = -a / b   # = -1.5
```

Целочисленное деление (округление «вниз»!):

```
a = 9; b = 6
x = 3 // 4    # = 0
x = a // b    # = 1
x = -3 // 4   # = -1
x = -a // b   # = -2
```

Остаток от деления

% – остаток от деления

```
d = 85
```

```
b = d // 10
```

```
a = d % 10
```

```
d = a % b
```

```
d = b % a
```

Для отрицательных чисел:

```
a = -7
```

```
b = a // 2 # -4
```

```
d = a % 2 # 1
```



Как в математике!

остаток ≥ 0

$$-7 = (-4) * 2 + 1$$

Сокращенная запись операций

$a += b$ $\#$ $a = a + b$

$a -= b$ $\#$ $a = a - b$

$a *= b$ $\#$ $a = a * b$

$a /= b$ $\#$ $a = a / b$

$a // = b$ $\#$ $a = a // b$

$a \% = b$ $\#$ $a = a \% b$

$a += 1$

увеличение на 1

Форматный вывод

```
a = 123
```

```
print ( "{:5d}".format(a) )
```

целое

__123

5 знаков

```
a = 5
```

```
print ( "{:5d}{:5d}{:5d}".format  
        (a, a*a, a*a*a) )
```

__5__25__125

5 знаков 5 знаков 5 знаков

Вещественные числа



Целая и дробная части числа разделяются точкой!

Форматы вывода:

```
x = 123.456
```

```
print( x )
```

```
print( "{:10.2f}".format(x) )
```

```
123.456
```

```
_____ 123.46
```

всего знаков

в дробной части

```
print( "{:10.2g}".format(x) )
```

```
___1.2e+02
```

значащих цифр

$1,2 \cdot 10^2$

Вещественные числа

Экспоненциальный формат:

```
x = 1. / 30000
```

```
print("{:e}".format(x))
```

```
x = 12345678.
```

```
print("{:e}".format(x))
```

$3,333333 \cdot 10^{-5}$

3.333333e-05

1.234568e+07

$1,234568 \cdot 10^7$

```
x = 123.456
```

```
print("{:e}".format(x))
```

```
print("{:10.2e}".format(x))
```

1.234560e+02

__1.23e+02

всего знаков

в дробной части

Стандартные функции

`abs(x)` — модуль числа

`int(x)` — преобразование к целому числу

`round(x)` — округление

`import math`

подключить
математический модуль

`math.pi` — число «пи»

`math.sqrt(x)` — квадратный корень

`math.sin(x)` — синус угла, заданного **в радианах**

`math.cos(x)` — косинус угла, заданного **в радианах**

`math.exp(x)` — экспонента e^x

`math.ln(x)` — натуральный логарифм

`math.floor(x)` — округление «вниз»

`math.ceil(x)` — округление «вверх»

```
x = math.floor(1.6) # 1
```

```
x = math.ceil(1.6) # 2
```

```
x = math.floor(-1.6) #-2
```

```
x = math.ceil(-1.6) #-1
```

Обработка цифр числа

При помощи операции нахождения остатка и целочисленного деления можно достаточно несложно вычислить любую цифру числа. Рассмотрим программу получения цифр **двузначного числа**:

```
num = 17  
a = num % 10  
b = num // 10  
print(a)  
print(b)
```

Результатом выполнения программы будут два числа:

7

1

Алгоритм получения цифр n-значного числа

Каждую цифру n-значного числа **num** можно найти, используя следующий алгоритм:

Последняя цифра: $(\text{num} \% 10^1) // 10^0;$

Предпоследняя цифра:

$(\text{num} \% 10^2) // 10^1;$

Предпредпоследняя цифра:

$(\text{num} \% 10^3) // 10^2;$

.....

Вторая цифра: $(\text{num} \% 10^{n-1}) // 10^{n-2};$

Первая цифра: $(\text{num} \% 10^n) // 10^{n-1}.$

Задача 1

Напишите программу, в которой рассчитывается сумма цифр двузначного числа.

Решение. Программа, решающая поставленную задачу, может иметь следующий вид:

```
num = int(input())  
last = num % 10  
first = num // 10  
print('Сумма цифр =', last + first)
```

Задача 2

Напишите программу, в которую вводится трёхзначное число и которая выводит на экран его цифры (через запятую).

Решение. Программа, решающая поставленную задачу, может иметь следующий вид:

```
num = int(input())
digit3 = num % 10
digit2 = (num // 10) % 10
digit1 = num // 100
print(digit1, digit2, digit3, sep=',')
```

Программирование на языке Python

Списки

Списки в Python

Списки представляют собой упорядоченные коллекции элементов. Они позволяют хранить в одном месте взаимосвязанные данные.

Списки (тип `list`) являются изменяемыми объектами

Например создадим список из 5 элементов и сохраним его в переменную `marks`.

```
marks = [4, 5, 4, 3, 5]  
print(type(marks))
```

Протокол вывода:

`<class 'list'>`

Списки в Python

Содержимое списка пишется в квадратных скобках, элементы списка разделяются запятой.

Списки могут хранить значение разных типов.

```
a = [True, 43, "text", 32.12, [2, 3, 4]]
```

У каждого элемента есть свой порядковый номер — **индекс**. С помощью индекса можно получить значение элемента списка.

Списки в Python

```
alphabet = ['а', 'б', 'в', 'г', 'д', 'ю', 'я']  
print(alphabet[1])  
print(alphabet[2])
```

Протокол вывода:

б

в



Счёт в списках начинается с нуля!

Списки в Python

```
alphabet = ['а', 'б', 'в', 'г', 'д', 'ю', 'я']  
print(alphabet[0])
```



Что получим?

Протокол вывода:
а

у первого элемента
индекс нулевой

Списки в Python

Можно сделать список из выражений, тогда в нём будут храниться вычисленные значения:

```
# сохраним в списках вторую и третью строки  
# таблицы Пифагора  
pithagoras_2 = [2*1,2*2,2*3,2*4,2*5,2*6,2*7,2*8,2*9]  
pithagoras_3 = [3*1,3*2,3*3,3*4,3*5,3*6,3*7,3*8,3*9]  
print(pithagoras_2)  
print(pithagoras_3)
```

Протокол вывода:

```
[2, 4, 6, 8, 10, 12, 14, 16, 18]  
[3, 6, 9, 12, 15, 18, 21, 24, 27]
```

Списки в Python

К списку, который хранится в переменной, можно прибавить другой список.

```
trubadur = ['Трубадур']  
animals = ['Кот', 'Пёс', 'Осёл', 'Петух']  
bremen_musicians = trubadur + animals  
print(bremen_musicians)
```

Протокол вывода:

['Трубадур', 'Кот', 'Пёс', 'Осёл', 'Петух']

Списки в Python

Для подсчёта элементов списка есть стандартная функция `len()`.

```
bremen_musicians = ['Трубадур', 'Кот',  
                    'Пёс', 'Осёл', 'Петух']  
  
count = len(bremen_musicians)  
  
print(count)
```

Протокол вывода:

5

Списки в Python

```
a = [1, 3] + [4, 23] + [5]  
print(a)
```

```
[1, 3, 4, 23, 5]
```



Что будет
напечатано?

```
a = [0]*10  
print(a)
```

```
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Позволяет
преобразовать в список

Диапазон от 0 до 10

```
a = list ( range(10) )  
print(a)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Метод join()

Сделать одну строку из списка строк позволяет метод join. Он принимает список как аргумент и возвращает одну строку, например:

```
slovo = ['с', 'п', 'и', 'с', 'к', 'и']  
print(' '.join(slovo))
```

Протокол вывода:
СПИСКИ

Списки в Python



Вызов метода записывают всегда так:
объект.имя_метода(аргументы_метода)

Например:

```
pushkin = ['божество', 'вдохновение', 'жизнь', 'слёзы', 'любовь']  
text = ', и '.join(pushkin)  
print('Что было после чудного мгновенья: и ' + text)
```

Протокол вывода:

Что было после чудного мгновенья: и божество, и
вдохновение, и жизнь, и слёзы, и любовь

Метод `.join()` сделал из списка строку, а объект, к которому применяли метод, стал разделителем.

В языке Python списки (тип `list`) являются:

Выберите один вариант из списка

- ☐ неизменяемыми
- ☒ изменяемыми

Что выведет следующий код?

```
zeros = [0] * 10  
print(len(zeros))
```

- ☐ 0
- ☐ `len(zeros)`
- ☐ Syntax Error
- ☒ 10

Что выведет следующий код?

```
words = ['Hello', 'Python']  
print('-'.join(words))
```

- ☒ Hello-Python
- ☐ HelloPython-
- ☐ -HelloPython
- ☐ -Hello-Python
- ☐ -Hello-Python-

Что выведет следующий код?

```
numbers = [1, 2, 3, 4, 5, 6, 7]  
print(numbers[5])  
print(numbers[0])  
print(numbers[7])
```

6

1

IndexError: list index out of range

Что выведет следующий код?

```
list1 = ['a', 'b']  
list2 = ['e', 'f']  
print(list1*2)  
print(list1+list2)
```

['a', 'b', 'a', 'b']
['a', 'b', 'e', 'f']

Что выведет следующий код?

```
numbers = [10, 20, 30, 40, 50, 60, 70, 80]  
print(len(numbers))
```

8

Программирование на языке Python

Ветвления

В Python есть логические выражения, которые возвращают ответ: истинно или ложно выражение.

Например:

`x = 5`

оператор присваивания:
переменной `x` присвоено
значение 5

`x == 5`

логический оператор
«равно» — это вопрос: «а
правда ли, что `x` равен
пяти?».

В зависимости от того, какое значение присвоено переменной `x`, программа даст ответ («вернёт значение»): «да, правда», `True`; или «нет, неправда», `False`.

Логические операторы

>

<

больше, меньше

>=

больше или равно

<=

меньше или равно

==

равно

!=

не равно

`5 < 3`: «а правда ли, что 5 меньше трёх?». Ответ: `False`.

`100 > 1`: «действительно ли сто больше одного?». Ответ: `True`.

Проверим, считает ли Python истинными выражения «дважды два — четыре» и «дважды два больше шести».

```
check = (2*2 > 6)  
print(check)
```

```
# Будет напечатано: False
```

```
check = (2*2 == 4)  
print(check)
```

```
# Будет напечатано: True
```

Ветвление кода

Ветвления позволяют писать код, который выполняется, когда логическое выражение истинно.

Ветвление объявляют оператором **if**

За ним идёт условие — логическое выражение, которое при выполнении программы примет значение True или False.

Если выражение в условии истинно, то выполняется блок инструкций после двоеточия. Если ложно, код после условия не сработает.

```
if <условие>:
```

```
    <код, который выполнится, если условие вернуло True>
```

Внимание: этот блок имеет отступ в 4 пробела от начала строки

Ветвление кода

Оператор **if** может включать условие «на все остальные случаи». Для этого существует конструкция **if / else**. Если условное выражение истинно, выполняется код блока **if**, а если ложно — код блока **else**.

```
if <условие>:
```

```
    <код, который выполнится, если условие истинно>
```

```
    # Внимание: этот блок имеет отступ в 4 пробела от  
    начала строки
```

```
else:
```

```
    <код, который выполнится, если условие ложно>
```

```
    # Этот блок тоже отбит от края строки 4 пробелами
```

Ветвление кода

Проанализируем скорость ветра по двенадцатибалльной [шкале Бофорта](#), используя ветвления в Python:

```
beaufort = 0
if beaufort == 0:
    print('Штиль')
else:
    print('Есть ветер')
```

Протокол вывода:

Штиль

Программирование на языке Python

**Вложенные условные
операторы, каскадное
ветвление, сложные условия**

Вложенные условные операторы

Задача: в переменных **a** и **b** записаны возрасты Андрея и Бориса. Кто из них старше?

?

Сколько вариантов?

```
if a == b:  
    print("Одного возраста")  
else:  
    if a > b:  
        print("Андрей старше")  
    else:  
        print("Борис старше")
```

?

Зачем нужен?

вложенный
условный оператор

Каскадное ветвление

```
if a == b:  
    print("Одного возраста")  
elif a > b:  
    print("Андрей старше")  
else:  
    print("Борис старше")
```

 `elif = else if`

Каскадное ветвление

```
cost = 1500
if cost < 1000:
    print ( "Скидок нет." )
elif cost < 2000:
    print ( "Скидка 2%." )
elif cost < 5000:
    print ( "Скидка 5%." )
else:
    print ( "Скидка 10%." )
```

первое сработавшее
условие



Что выведет?

Скидка 2%.

Сложные условия

Задача: набор сотрудников в возрасте **25-40 лет**
(включительно).

сложное условие

```
if v >= 25 and v <= 40 :  
    print("подходит")  
else:  
    print("не подходит")
```

and «И»

or «ИЛИ»

not «НЕ»

Приоритет :

- 1) отношения (<, >, <=, >=, ==, !=)
- 2) **not** («НЕ»)
- 3) **and** («И»)
- 4) **or** («ИЛИ»)

Программирование на языке Python

Циклы

Цикл — это многократное выполнение одинаковых действий.

Два вида циклов:

- цикл с **известным** числом шагов (сделать 10 раз)
- цикл с **неизвестным** числом шагов (делать, пока выполняется условие)

Объявление цикла

Чтобы программа поняла, что сейчас начнётся цикл — нужно сообщить ей об этом специальными словами: **объявить цикл**.

Цикл в Python объявляется ключевыми словами **for** и **in**, после них задаётся **условие цикла**. После условия ставится двоеточие.

```
for переменная in список_элементов:
```

В условии цикла после **for** указывают имя переменной, в которую будут поочерёдно передаваться элементы из обрабатываемого списка, а после **in** ставится имя списка, который надо обработать.

Тело цикла

За условием с новой строки следует **тело цикла**. Каждая строка тела цикла отбивается от начала строки четырьмя пробелами:

```
musicians = ['Кот', 'Пёс', 'Осёл', 'Петух']  
for musician in musicians:  
    # Каждый элемент списка musicians  
    # по очереди будет передан в переменную musician  
    # и напечатан  
    print(musician)  
# Код, который выполняется после цикла  
print('Нам дворцов заманчивые своды не заменят никогда  
свободы!')
```

Цикл бежит по кругу, пока не переберёт все элементы списка. После этого выполнится код, печатающий строку из песни.

Каждый «круг» цикла называется **итерацией цикла**.

Отступы перед вложенным кодом

Код, который размещён в теле цикла, отбивается от начала строки четырьмя отступами.

По этим отступам Python определяет, где начинается и кончается код, относящийся к телу цикла.

Это строгое техническое условие: без отступов в блоках кода программа просто не заработает.

Программирование на языке Python

Диапазоны

Диапазоны

В Python есть функция **range()**. В неё передаются два целых числа: начало и конец диапазона. В результате будет создана последовательность, включающая все целые числа в указанном диапазоне.

Диапазоны

Особенность этой последовательности в том, что в неё не включается последнее число диапазона:

```
three = range(0, 3)
# Последовательность three будет включать в
# себя числа 0, 1 и 2.
```

Числа могут быть и отрицательными, важно лишь, чтобы первое число было меньше второго.

```
around_zero = range(-3, 3)
# Последовательность around_zero будет
# включать в себя числа -3, -2, -1, 0, 1 и 2.
```

Если в `range()` указать только одно число — программа воспримет это число как конец диапазона чисел, начинающегося с 0.

`range(0, 10)` и `range(10)` — это одно и то же.

Диапазоны

Особенность последовательности `range()` состоит в том, что её можно обрабатывать в цикле, как обычный список.

Например:

```
around_zero = range(-3, 3)
# Вместо списка в цикл передаётся переменная
# в ней хранится range() от -3 до 3
for i in around_zero:
    # Перебрать все числа в диапазоне от -3 до 3
    print(i)
# и напечатать их:
```

Протокол вывода:

-3
-2
-1
0
1
2

Диапазоны

Задать диапазон можно прямо в условии цикла, без промежуточной переменной `around_zero`.

```
# Цикл переберёт все числа в диапазоне от -3
# до 3 и напечатает их:
for i in range(-3, 3):
    print(i)
# Результат будет тот же
```

Последовательность в обратном порядке

Функция `reversed()` может развернуть в обратном направлении любую последовательность:

```
# Можно обратить вспять обычный список:  
seasons = ['зима', 'весна', 'лето', 'осень']  
for season in reversed(seasons):  
    # Переменную цикла, в которую будут  
    # передаваться элементы "перевёрнутого"  
    # списка seasons, назовём season  
    print(season)
```

Протокол вывода:

осень

лето

весна

зима

Цикл с переменной: другой шаг

10, 9, 8, 7, 6, 5, 4, 3, 2, 1

шаг

```
for k in range(10, 0, -1):
    print ( k**2 )
```



Что получится?

1, 3, 5, 7, 9

```
for k in range(1, 11, 2):
    print ( k**2 )
```

100

81

64

49

36

25

16

9

4

1

1

9

25

49

81

Какое значение примет переменная a?

```
a = 1
```

```
for i in range(3): a += 1
```

a = 4

```
a = 1
```

```
for i in range(3, 1): a += 1
```

a = 1

```
a = 1
```

```
for i in range(1, 3, -1): a += 1
```

a = 1

```
a = 1
```

```
for i in range(3, 1, -1): a += 1
```

a = 3

Программирование на языке Python

Цикл с условием.

Цикл с условием

Задача. Определить **количество цифр** в десятичной записи целого положительного числа, записанного в переменную n .

```
счётчик = 0
```

```
пока  $n > 0$ :
```

```
    отсечь последнюю цифру  $n$   
    увеличить счётчик на 1
```

n	счётчик
-----	---------

1234	0
------	---

?

Как отсечь последнюю цифру?

```
 $n = n // 10$ 
```

?

Как увеличить счётчик на 1?

```
счётчик = счётчик + 1
```

```
счётчик += 1
```

Цикл с условием

начальное значение
счётчика

условие
продолжения

заголовок
цикла

```
count = 0  
while n > 0 :
```

```
    n = n // 10  
    count += 1
```

тело цикла



Цикл с предусловием – проверка на входе в цикл!

Цикл с условием

При известном количестве шагов:

```
k = 0
while k < 10:
    print ( "привет" )
    k += 1
```

Заикливание:

```
k = 0
while k < 10:
    print ( "привет" )
```

Цикл с постусловием

Задача. Обеспечить ввод **положительного** числа в переменную `n`.

бесконечный
цикл

```
while True:
```

```
    print ( "Введите положительное число:" )
```

```
    n = int ( input() )
```

```
    if n > 0: break
```

тело цикла

условие
выхода

прервать
цикл

- при входе в цикл условие **не проверяется**
- цикл всегда выполняется **хотя бы один раз**

Прерывания цикла while

Break — ключевое слово `break` прерывает цикл и передает управление в конец цикла

```
a = 1
while a < 5:
    a += 1
    if a == 3:
        break
print(a)
```



Что получится?

2

Прерывания цикла while

Continue — ключевое слово, прерывает текущую итерацию и передает управление в начало цикла, после чего условие снова проверяется. Если оно истинно, выполняется следующая итерация.

```
a = 1
while a < 5:
    a += 1
    if a == 3:
        continue
    print(a)
```



Что получится?

2
4
5

Задача. Вывести 10 раз слово «Привет!».

Цикл с переменной:

```
for i in range(10):  
    print("Привет!")
```

в диапазоне
[0,10)



Можно ли сделать с циклом «пока»?

Цикл с условием:

```
i = 0  
while i < 10:  
    print("Привет!")  
    i += 1
```

Задача. Вывести все степени двойки от 2^1 до 2^{10} .

Цикл с переменной:

```
for k in range(1, 11) :  
    print ( 2**k )
```

в диапазоне
[1, 11)



Как сделать с циклом «пока»?

Цикл с условием:

```
k = 1  
while k <= 10 :  
    print ( 2**k )  
    k += 1
```

Задача. Вывести квадраты чисел: 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

Цикл с переменной:

шаг



Что получится?

```
for k in range(10, 0, -1):  
    print ( k**2 )
```

100
81
64
49
36
25
16
9
4
1

Цикл с условием:

```
k = 10  
while k > 0 :  
    print (k**2)  
    k -= 1
```

Задача. Вывести квадраты нечетных чисел от 1 до 10.

Цикл с переменной:

```
for k in range(1, 11, 2) :  
    print ( k**2 )
```



Что получится?

1
9
25
49
81

Цикл с условием:

```
k = 1  
while k < 10 :  
    print (k**2)  
    k += 2
```

Инструкции управления циклом While

После тела цикла можно написать слово `else`: и после него блок операций, который будет выполнен *один раз* после окончания цикла, когда проверяемое условие станет неверно:

```
i = 1
while i <= 10:
    print(i)
    i += 1
else:
    print('Цикл окончен, i =', i)
```

Пример программы, которая считывает числа до тех пор, пока не встретит отрицательное число. При появлении отрицательного числа программа завершается. Последовательность чисел завершается числом 0 (при считывании которого надо остановиться).

```
a = int(input())
while a != 0:
    if a < 0:
        print('Встретилось отрицательное число', a)
        break
    a = int(input())
else:
    print('Ни одного числа < 0 не встретилось')
```

Во втором варианте программы сначала на вход подается количество элементов последовательности, а затем и сами элементы. В таком случае удобно воспользоваться циклом for. Цикл for также может иметь ветку else и содержать инструкции break внутри себя.

```
n = int(input())
for i in range(n):
    a = int(input())
    if a < 0:
        print('Встретилось отрицательное число', a)
        break
else:
    print('Ни одного числа < 0 не встретилось')
```


Вложенные циклы

Задача. Вывести все простые числа в диапазоне от 2 до 1000.

сделать для n от 2 до 1000
если число n простое то
вывод n

нет делителей $[2.. n-1]$:
проверка в цикле!

 Что значит «простое число»?

```
for n in range(2, 1001):  
    if число n простое:  
        print( n )
```

Вложенные циклы

```
for n in range(2, 1001):  
    count = 0  
    for k in range(2, n):  
        if n % k == 0:  
            count += 1  
    if count == 0:  
        print( n )
```

ВЛОЖЕННЫЙ ЦИКЛ

Вложенные циклы

```
for i in range(1,4):  
    for k in range(1,4):  
        print( i, k )
```

i k

1 1

1 2

1 3

2 1

2 2

2 3

3 1

3 2

3 3



Как меняются переменные?



Переменная внутреннего цикла изменяется быстрее!

Вложенные циклы

```
for i in range(1,5):  
    for k in range(1,i+1):  
        print( i, k )
```

i k

1 1

2 1

2 2

3 1

3 2

3 3

4 1

4 2

4 3

4 4



Как меняются переменные?



Переменная внутреннего цикла изменяется быстрее!

Сколько раз выполняется цикл?

```
a = 4; b = 6  
while a < b: a += 1
```

2 раза
a = 6

```
a = 4; b = 6  
while a < b: a += b
```

1 раз
a = 10

```
a = 4; b = 6  
while a > b: a += 1
```

0 раз
a = 4

```
a = 4; b = 6  
while a < b: b = a - b
```

1 раз
b = -2

```
a = 4; b = 6  
while a < b: a -= 1
```

зацикливание



Программирование на языке Python

Случайные числа

Случайные числа

Случайно...

- встретить друга на улице
- разбить тарелку
- найти 10 рублей
- выиграть в лотерею

Случайный выбор:

- жеребьевка на соревнованиях
- выигравшие номера в лотерее

Как получить случайность?



Случайные числа на компьютере

Псевдослучайные числа – обладают свойствами случайных чисел, но каждое следующее число вычисляется по заданной формуле.

Метод середины квадрата (Дж. фон Нейман)

зерно

564321

в квадрате

малый период
(последовательность
повторяется через 10^6 чисел)

318458191041

209938992481

Генератор случайных чисел

```
import random
```

англ. *random* – случайный

Целые числа на отрезке [a,b]:

```
X = random.randint(1, 6) # псевдосл. число  
Y = random.randint(1, 6) # уже другое!
```

Генератор на [0,1):

```
X = random.random() # псевдослучайное число  
Y = random.random() # это уже другое число!
```

Генератор на [a, b] (вещественные числа):

```
X = random.uniform(1.2, 3.5)  
Y = random.uniform(1.2, 3.5)
```

Генератор случайных чисел

```
from random import *
```

Целые числа на отрезке [a,b]:

```
X = randint(10, 60) # псевдослучайное число  
Y = randint(10, 60) # это уже другое число!
```

Генератор на [0,1):

```
X = random() # псевдослучайное число  
Y = random() # это уже другое число!
```

Генератор на [a, b] (вещественные числа):

```
X = uniform(1.2, 3.5) # псевдосл. число  
Y = uniform(1.2, 3.5) # уже другое число!
```

Программирование на языке Python

Функции

Что такое функция?

Функция — это именованный блок кода, выполняющий определённую задачу.

Код функции можно использовать многократно, надо лишь вызвать её — обратиться к ней по имени.

В Python есть множество встроенных функций
Например: `print()`, `str()`, `int()`, `float()`, `len()`.

Но можно создавать и собственные функции.

Объявление функции

Всё начинается с **объявления функции**, со строки, которая означает «здесь мы создаём новую функцию».

Функцию объявляют ключевым словом `def`, затем указывают **имя** функции (это имя придумывает разработчик), после имени — круглые скобки и двоеточие.

За объявлением функции следует код, который функция должна выполнить при вызове. Этот код называется **тело функции**;

Тело функции отбивается четырьмя пробелами от начала строки: по этим отступам Python понимает, где начинается и заканчивается тело функции. Нет отступов — нет функции.

Объявим функцию, которая будет приветствовать разработчика:

```
# Объявление функции hello()
def hello():
    # А здесь началось тело функции
    print('Приветствую тебя!')
```



Что произойдет,

Но если выполнить этот код — ничего не произойдёт. Совсем-совсем ничего. Функция не выполнится до тех пор, пока где-то в коде программы не будет **вызова функции**, команды «функция, делай свою работу!».

Вызов функции

Пока функция не вызвана — она не выполняется: она просто лежит и ждёт своего часа.

Функция вызывается по имени, после имени ставятся круглые скобки.

```
def hello():  
    print('Приветствую тебя!')  
hello()  
hello()  
hello()
```

Приветствую тебя!
Приветствую тебя!
Приветствую тебя!

Параметры и аргументы функции

При объявлении функции можно указывать **параметры функции** — переменные, которые будут обрабатываться в её теле. Имена для параметров придумывает сам разработчик.

Значения для этих параметров передаются при вызове; передаваемые при вызове значения называют **аргументами функции**.

Параметры указывают в круглых скобках при объявлении функции; аргументы указывают при вызове функции.

Параметры и аргументы функции

1

2

3

1

ключевое слово

2

имя функции

3

параметры функции,
в скобках

```
def hello(name):
```

```
    #Началось тело функции с 4 отступами
```

```
    print(name+ ' , Приветствую тебя!')
```

```
    # В теле функции может быть еще много кода
```

```
#Тело функции кончилось, когда начался код без
```

```
#отступов
```

```
#Вызов функции
```

```
hello('Максим')
```

```
# Будет напечатано: Максим, приветствую тебя!
```

4

4

Аргумент, передаваемый при
вызове функции

Параметры и аргументы функции

```
def hello(name):  
    print(name+' , Приветствую тебя!')  
hello('Максим')  
hello('Иван')  
hello('Олег')
```

Протокол вывода:
Максим, Приветствую тебя!
Иван, Приветствую тебя!
Олег, Приветствую тебя!

Параметры и аргументы функции

У функции может быть и несколько параметров. При вызове функции аргументы передаются в соответствии с порядком записи: первый аргумент передаётся в первый параметр, второй аргумент — во второй параметр.

```
# Теперь у функции hello() два параметра: name и bonus
def hello(name, bonus):
    # Оба параметра применим в теле функции:
    print(name + ', приветствую тебя! Бери ' + bonus)
hello('Дарт Вейдер', 'печеньки')
```

Параметры и аргументы функции

```
def hello(name, bonus):
```

```
    print(name + ', приветствую тебя! Бери ' + bonus)
```

```
# Вызов функции
```

```
hello('Дарт Вейдер', 'печеньки')
```

```
# Будет напечатано:
```

```
# Дарт Вейдер, приветствую тебя! Бери печеньки
```

Параметры и аргументы функции

```
def hello(name, bonus):  
    print(name + ', приветствую тебя! Бери ' + bonus)  
hello('Дарт Вейдер', 'печеньки')  
hello('Винни Пух', 'мёд')
```

Протокол вывода:

Дарт Вейдер, Приветствую тебя! Бери печеньки

Винни Пух, Приветствую тебя! Бери мёд

Аргумент функции

```
def hello(name, bonus):  
    print(name + ', приветствую тебя! Бери ' + bonus)  
  
hello('Пьер')
```

! Traceback (most recent call last):

File "C:/Users/Домашний/AppData/Local/Programs/Python/Python39/1.py", line 3, in <module>

hello('Пьер')

TypeError: hello() missing 1 required positional argument: 'bonus'

Всё сломалось:

Потерялся обязательный позиционный аргумент **'bonus'**

Значение по умолчанию

При объявлении функции любому параметру можно присвоить «значение по умолчанию»: это значение будет передано параметру, если при вызове функции ожидаемый аргумент не был получен.

При вызове не передан второй аргумент, однако функция отработает без ошибок:

```
def hello( name='Инкогнито' , bonus ='кекс') :  
    print(name + ' , приветствую тебя! Бери ' + bonus)  
hello('Пьер')
```

Пьер, приветствую тебя! Бери кекс

Именованные и позиционные аргументы

При вызове функции значения передаются в параметры соответственно с их позицией, в порядке их перечисления: первый аргумент передаётся в первый параметр, второй аргумент — во второй параметр.

Это называется **позиционные аргументы**.

```
def print_home(name='Инкогнито', planet='Икс'):  
    print(name + ' живёт на планете ' + planet)  
print_home('Земля')
```

Земля живет на планете Икс

?

Результат?

?

Как избежать?

Именованные и позиционные аргументы

Чтобы избежать несоответствия аргументов параметрам, при вызове функции нужно передавать **именованные аргументы** — явно указывать, какому параметру какой аргумент соответствует.

```
def print_home(name='Инкогнито', planet='Икс'):  
    print(name + ' живёт на планете ' + planet)  
# Передаём именованный параметр:  
# явно указываем, что значение 'Земля' предназначено  
# для параметра planet  
print_home(planet='Земля')  
# Ещё раз вызовем функцию: передадим два именованных  
# параметра,  
print_home(planet='Марс', name='Марк Уотни')
```

Инкогнито живёт на планете Земля
Марк Уотни живёт на планете Марс

Задача. Написать функцию, которая вычисляет сумму цифр числа

```
сумма = 0
пока n != 0:
    сумма += n % 10
    n = n // 10
```

Сумма цифр числа

```
def sumDigits( n ):  
    sum = 0  
    while n != 0:  
        sum += n % 10  
        n = n // 10  
    return sum
```

передача
результата



Чего не
хватает?

```
# основная программа  
sumDigits(12345)
```

```
# сохранить в переменной  
n = sumDigits(12345)
```

```
# сразу вывод на экран  
print ( sumDigits(12345) )
```

Использование функций

```
x = 2*sumDigits (n+5)
z = sumDigits (k) + sumDigits (m)
if sumDigits (n) % 2 == 0:
    print ( "Сумма цифр чётная" )
    print ( "Она равна", sumDigits (n) )
```



Функция, возвращающая целое число, может использоваться везде, где и целая величина!

Программирование на языке Python

**Функции. Локальные и
глобальные переменные**

Локальные и глобальные переменные

глобальная
переменная

локальная
переменная

```
a = 5
def qq():
    a = 1
    print(a)
qq()
print(a)
```

1

5

```
a = 5
def qq():
    print(a)
qq()
```

5

```
a = 5
def qq():
    global a
    a = 1
qq()
print(a)
```

работаем с
глобальной
переменной

1

Неправильная процедура

```
x = 5; y = 10  
  
xSum()
```



Что плохо?

```
def xSum():  
    print(x+y)
```

- 1) процедура связана с глобальными переменными, нельзя перенести в другую программу
- 2) печатает только сумму x и y , нельзя напечатать сумму других переменных или сумму $x*y$ и $3x$



Как исправить?

передавать
данные через
параметры

Правильная процедура

Глобальные:

x	y
5	10
z	w
17	3

```
x = 5; y = 10
Sum2 ( x, y )
z=17; w=3
Sum2 ( z, w )
Sum2 ( z+x, y*w )
```

```
def Sum2 (a, b) :
    print ( a+b )
```

Локальные:

a	b	
25	30	15
		20
		52



- 1) процедура не зависит от глобальных переменных
- 2) легко перенести в другую программу
- 3) печатает сумму любых выражений

Как вернуть несколько значений?

```
def divmod ( x, y ) :  
    d = x // y  
    m = x % y  
    return d, m
```

d – частное,
m – остаток

```
a, b = divmod ( 7, 3 )  
print ( a, b )           # 2 1
```

кортеж – набор
элементов

```
q = divmod ( 7, 3 )  
print ( q )              # (2, 1)
```

q[0]

q[1]

Решение задач

1

Напишите функцию `digit_n`, которая принимает два параметра: `digit` – цифру в строковом представлении и `n` – количество цифр и печатает переданную цифру `n` раз в одну строку без пробела.

параметры

```
def digit_n (digit, n):  
    print (digit * n)  
# вызовы функции с аргументами  
digit_n("1", 2)  
digit_n("2", 8)  
digit_n("3", 4)
```

аргументы

Результат:

```
11  
22222222  
3333
```

Решение задач

2

Напишите функцию **maximum**, которая принимает два числа и возвращает большее из них. Организуйте ввод чисел с клавиатуры и вызовите функцию с ними в качестве аргументов.

```
def maximum (a, b) :  
    if a > b :  
        m = a  
    else:  
        m = b  
    return m  
x = int(input("Введите 1 число:"))  
y = int(input("Введите 2 число:"))  
print("max =", maximum(x, y))
```

Результат:
Введите 1 число: 24
Введите 2 число: 13
max = 24

Решение задач

3

Найти наибольший общий делитель двух чисел, используя в качестве процедуры алгоритм Евклида.

```
def nod(a, b):  
    while a != b:  
        if a > b:  
            a = a - b  
        else:  
            b = b - a  
    return a  
x = int(input("Введите 1 число:"))  
y = int(input("Введите 2 число:"))  
print("НОД = ", nod(x, y))
```

Результат:
Введите 1 число: 48
Введите 2 число: 32
НОД = 16

Решение задач

4

Напишите функцию `count_space` для нахождения количества пробелов в строке. Используйте ее для подсчета количества слов в введенном тексте.

```
def count_space (stroka):  
    k = 0 # нач. знач. счетчика  
    for c in stroka: # перебор символов  
        if c == " ": # если символ - пробел  
            k = k+1  
    return k  
a = input("Введите текст: ")  
print("В тексте ", count_space(a)+1, "слов(a) ")
```

Результат:

Введите текст: Мама мыла раму
В тексте 3 слов(a)

Решение задач

5

Удалить все пробелы в тексте, используя функцию `del_char` для удаления символов в строке.

СИМВОЛ

```
def del_char (stroka, y):  
    z = ""                # нач. знач. результата  
    for c in stroka :    # перебор символов строки  
        if c != y:  
            z = z + c  
    return z              # результат функции  
a = input("Введите текст:")  
b = del_char(a, " ")  
print("Изменённый текст:", b)
```

Результат:

Введите текст: Мама мыла раму
Изменённый текст: Мамамылараму

Решение задач

Составить программу для вычисления числа сочетаний из n по k .

В комбинаторике набор k элементов, выбранных из данного множества, содержащего n различных элементов, называется сочетанием из n по k . Значение этой величины вычисляется по формуле:

$$C_n^k = \frac{n!}{k!(n-k)!}$$



Какая функция поможет решить поставленную задачу?

Решение задач

6

$$C_n^k = \frac{n!}{k!(n-k)!}$$

```
def fact(x) :  
    p = 1          # нач. знач. произведения  
    for i in range(1, x+1):  
        p = p*i  
    return p  
  
n = int(input("Введите n: "))  
k = int(input("Введите k: "))  
c = fact(n) // (fact(k) * fact(n-k))  
print("Число сочетаний равно", c)
```

Результат:

Введите n: 7

Введите k: 5

Число сочетаний равно 21

Логические функции

Задача. Найти все простые числа в диапазоне от 2 до 1000.

```
for i in range(2, 1001):  
    if isPrime(i):  
        print ( i )
```

функция,
возвращающая
логическое значение
(True/False)

Функция: простое число или нет?

? Какой алгоритм?

```
def isPrime ( n ) :
```

```
    k = 2
```

```
    while k*k <= n and n % k != 0 :
```

```
        k += 1
```

```
    return (k*k > n)
```

```
        if k*k > n:
```

```
            return True
```

```
        else:
```

```
            return False
```

Перебирать надо только числа, не превосходящие корня из искомого. К примеру, если число M имеет делитель p_i , то имеется делитель q_i , такой, что $p_i * q_i = M$. То есть, чтобы найти пару, достаточно найти меньшее. Среди всех пар, предполагаемая пара с максимальным наименьшим — это пара с равными p_i и q_i , то есть $p_i * p_i = M \Rightarrow p_i = \sqrt{M}$.

Логические функции: использование



Функция, возвращающая логическое значение, может использоваться везде, где и логическая величина!

```
n = int ( input() )  
if isPrime(n):  
    print ( n, "- простое число" )
```

```
n = int ( input() )  
while isPrime(n):  
    print ( n, "- простое число" )  
    n = int ( input() )
```
